МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Моделирование сред и разработка приложений виртуальной и дополненной реальности»

Практическая работа № 3

Студент группы *ИКБО-50-23, Враженко Д.О.* _____
(подпись)

Старший преподаватель *Горбатов Г.В.* _____
(подпись)

Отчет представлен «___»_____202__г.

Москва 2026 г.

1. Создание интерактивного пользовательского интерфейса: окна, меню, UI

На рисунке 1 представлена сцена «Главное меню», на рисунке 2 показано как она будет выглядеть в игре, а в листинге 1 написан код скрипта для выполнения определенных действий при нажатии на соответствующие кнопки.

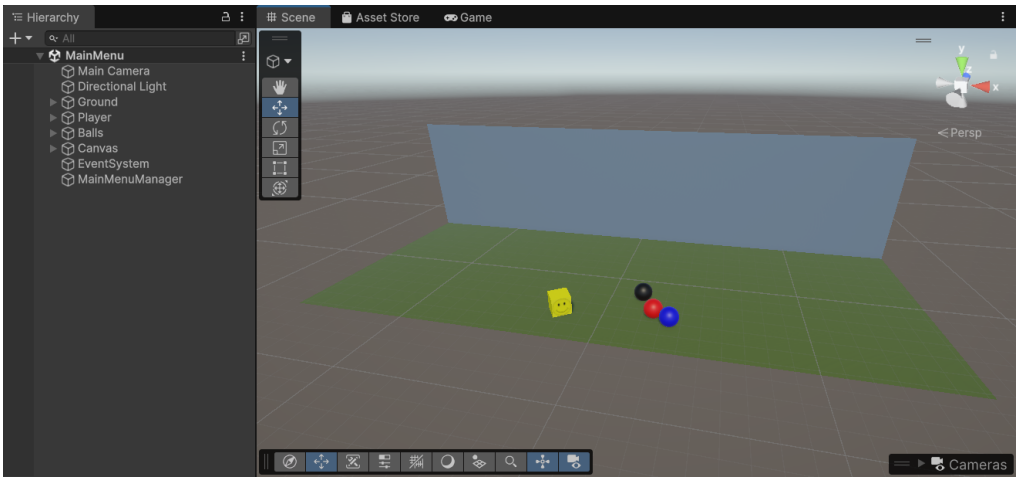

Рисунок 1 — Сцена «Главное меню»

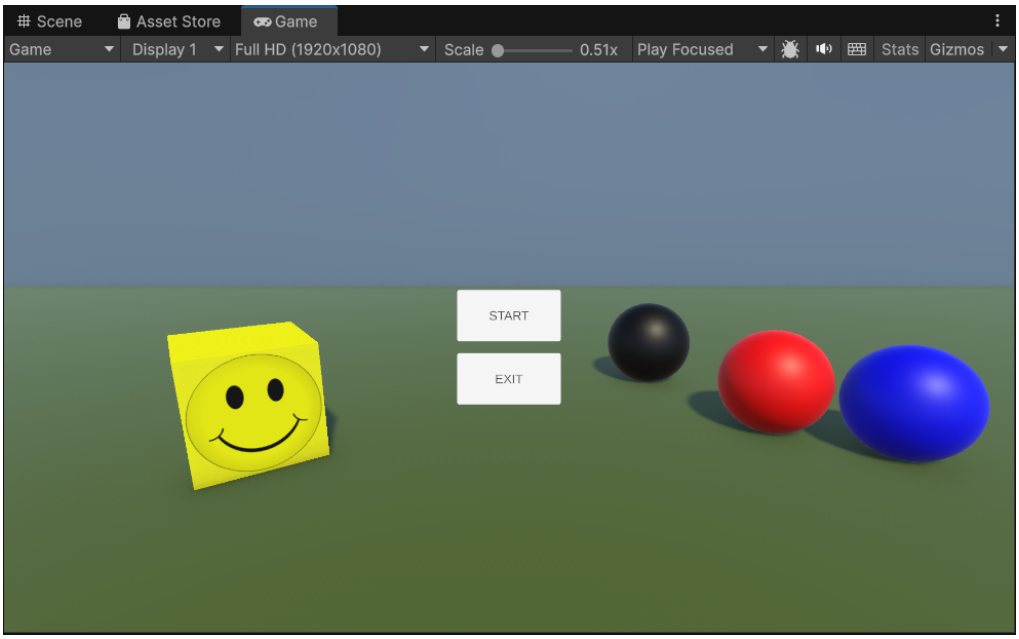

Рисунок 2 — Сцена «Главное меню» во время игры

Листинг 1 - Код скрипта MainMenuController.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class MainMenuController : MonoBehaviour
{
    public void StartGame()
```

```
    {
        SceneManager.LoadScene("GameScene");
    }

    public void ExitGame()
    {
        Application.Quit();
    }
}
```

2. Создание интерактивного визуального интерфейса: бары, показатели, GUI и HUD

Во время игры пользователь может отслеживать своё местоположение с помощью показателей высоты, на которых отображается информация о текущей высоте и максимальной, на которую когда-либо поднимался пользователь. Данный функционал продемонстрирован на рисунке 3, а в листинге 2 — код скрипта, отвечающего за изменения показателей.

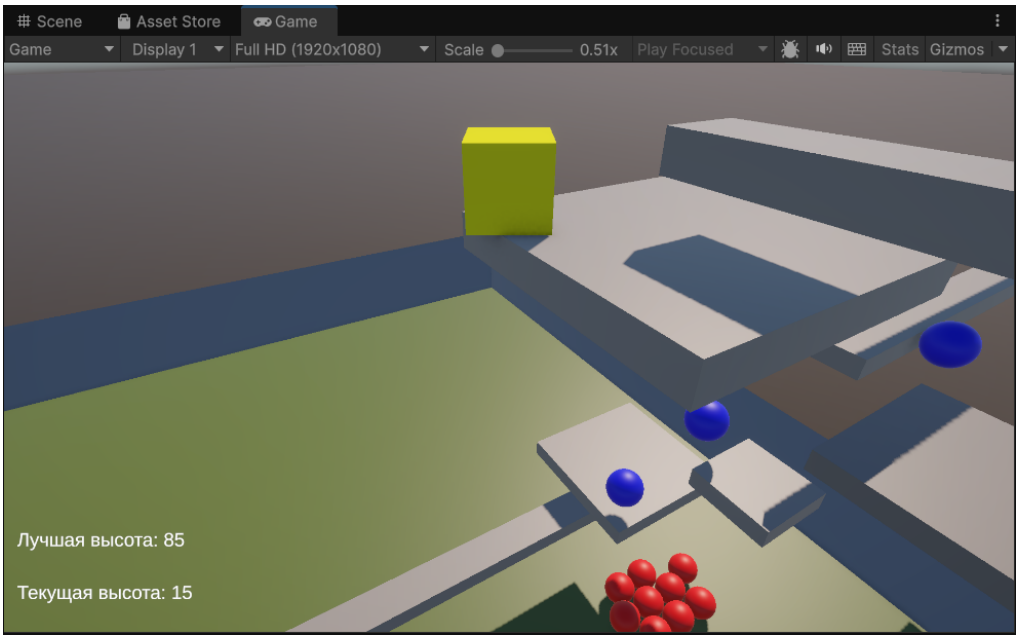

Рисунок 3 — Показатели высоты

Листинг 2 - Код скрипта HeightTracker.cs

```
using UnityEngine;
using TMPro;

public class HeightTracker : MonoBehaviour
{
    public Transform player;

    public TextMeshProUGUI bestText;
    public TextMeshProUGUI currentText;

    float currentHeight;

    void Start()
```

```
    {
        GameData.bestHeight = PlayerPrefs.GetFloat("BestHeight", 0f);
    }

    void Update()
    {
        if (player == null) return;
        currentHeight = player.position.y;
        if (currentHeight > GameData.bestHeight)
        {
            GameData.bestHeight = currentHeight;
            PlayerPrefs.SetFloat("BestHeight", GameData.bestHeight);
        }
        UpdateUI();
    }

    void UpdateUI()
    {
        bestText.text = "Лучшая высота: " +
Mathf.FloorToInt(GameData.bestHeight).ToString();
        currentText.text = "Текущая высота: " +
Mathf.FloorToInt(currentHeight).ToString();
    }
}
```

3. Настройка, добавление и смена сцены. Передача данных между сценами

Во время игры пользователь может нажать на кнопку «Escape», тем самым поставив игровой процесс на паузу. Здесь он может продолжить прохождение текущего сгенерированного уровня, начать игру заново или выйти в главное меню. Это продемонстрировано на рисунке 4. В листинге 3 приведен код скрипта для выполнения действий при нажатии кнопок.

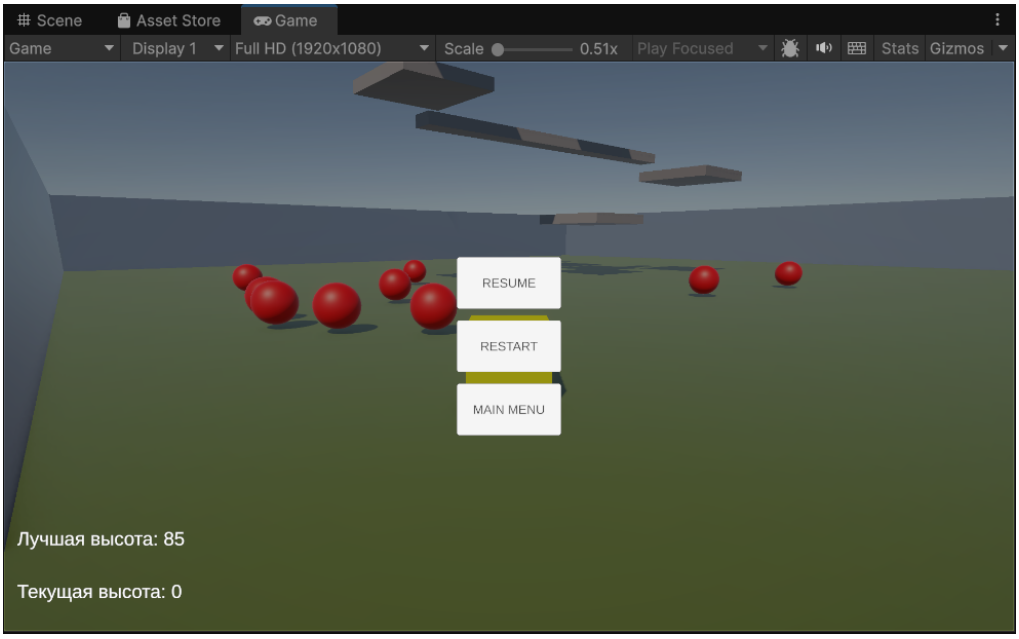

Рисунок 4 — Экран паузы

Листинг 3 - Код скрипта PauseMenuController.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class PauseMenuController : MonoBehaviour
{
    public GameObject pauseMenu;

    bool isPaused = false;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
            if (isPaused) Resume();
            else Pause();
    }

    public void Pause()
    {
        pauseMenu.SetActive(true);
        Time.timeScale = 0f;
        isPaused = true;
        Cursor.lockState = CursorLockMode.None;
        Cursor.visible = true;
    }

    public void Resume()
    {
        pauseMenu.SetActive(false);
        Time.timeScale = 1f;
        isPaused = false;
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    public void Restart()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }

    public void LoadMainMenu()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene("MainMenu");
    }
}
```

В листинге 2 уже был продемонстрирован функционал передачи данных между сценами. Для этого используются PlayerPrefs, в которых хранится информация о максимальной высоте пользователя. При запуске нового уровня оттуда считывается информация, а когда пользователь преодолевает свой рекорд, то хранящиеся данные заменяются на новые.